



Compiler Construction
Lab No. 8

Submitted By: Fatima Zahra
Sap Id: 55551
Submitted To: Ma'am Hasnat Bibi

Riphah International University of Islamabad

Task 1:

Given grammar:

$S \rightarrow A B$

$A \rightarrow aA \mid \varepsilon$

$B \rightarrow bB \mid \varepsilon$

1. Find **FIRST and FOLLOW sets**.

2. Apply **Intersection Method**.

Solution:

Step 1: Compute FIRST Sets

$\text{FIRST}(A) = \{a, \varepsilon\}$

- $A \rightarrow aA$ gives 'a'
- $A \rightarrow \varepsilon$ gives ε

$\text{FIRST}(B) = \{b, \varepsilon\}$

- $B \rightarrow bB$ gives 'b'
- $B \rightarrow \varepsilon$ gives ε

$\text{FIRST}(S) = \text{FIRST}(A) \cup \text{FIRST}(B)$ (since A can be ε)

- $\text{FIRST}(A)$ without $\varepsilon = \{a\}$
- Since A can be ε , add $\text{FIRST}(B) = \{b\}$
- $\text{FIRST}(S) = \{a, b\}$

Step 2: Compute FOLLOW Sets

$\text{FOLLOW}(S) = \{\$ \}$ (start symbol always gets \$)

$\text{FOLLOW}(A) = \text{FIRST}(B) = \{b\}$

- Since A is followed by B in $S \rightarrow A B$
- And B can be ε , also need to add $\text{FOLLOW}(S)$ when $B \rightarrow \varepsilon$
- So $\text{FOLLOW}(A) = \{b, \$ \}$

$\text{FOLLOW}(B) = \text{FOLLOW}(S) = \{\$ \}$

- B is at the end of production $S \rightarrow A B$

Final FOLLOW Sets:

$\text{FOLLOW}(S) = \{\$ \}$

$\text{FOLLOW}(A) = \{b, \$ \}$

$\text{FOLLOW}(B) = \{\$ \}$

Step 3: Apply Intersection Method

For $S \rightarrow A B$:

- Only one production, no conflict

$\text{FIRST}(aA) = \{a\}$

$\text{FIRST}(\varepsilon) = \{\varepsilon\}$

Check Rule 2 (since ε exists):

$\text{FIRST}(aA) \cap \text{FOLLOW}(A) = \{a\} \cap \{b, \$ \} = \emptyset$

Condition satisfied

For $B \rightarrow bB \mid \varepsilon$:

$\text{FIRST}(bB) = \{b\}$

$\text{FIRST}(\varepsilon) = \{\varepsilon\}$

Check Rule 2:

$\text{FIRST}(bB) \cap \text{FOLLOW}(B) = \{b\} \cap \{\$ \} = \emptyset$

Condition satisfied

Conclusion: The grammar is **LL(1)**.

Task 1 Results Table

Non-Terminal	FIRST Set	FOLLOW Set
S	{a, b}	{ \$ }
A	{a, ε }	{b, \$ }

B	{b, ε}	{\$}
----------	--------	------

Task 2:

For grammar:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow id \mid (E)$

Solution:

Perform:

1. Parse the string:

id + id * id

Show all stack operations.

Parsing Table (from lab manual):

Non-Terminal	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Step-by-Step Parsing of "id + id * id \$"

Step	Stack	Input	Action
1	\$ E	id + id * id \$	Start
2	\$ E' T	id + id * id \$	$E \rightarrow T E'$
3	\$ E' T' F	id + id * id \$	$T \rightarrow F T'$
4	\$ E' T' id	id + id * id \$	$F \rightarrow id$
5	\$ E' T'	+ id * id \$	Match id
6	\$ E'	+ id * id \$	$T' \rightarrow \epsilon$
7	\$ E' T +	+ id * id \$	$E' \rightarrow + T E'$
8	\$ E' T	id * id \$	Match +
9	\$ E' T' F	id * id \$	$T \rightarrow F T'$
10	\$ E' T' id	id * id \$	$F \rightarrow id$
11	\$ E' T'	* id \$	Match id
12	\$ E' T' F *	* id \$	$T' \rightarrow * F T'$
13	\$ E' T' F	id \$	Match *
14	\$ E' T' id	id \$	$F \rightarrow id$
15	\$ E' T'	\$	Match id
16	\$ E'	\$	$T' \rightarrow \epsilon$
17	\$	\$	$E' \rightarrow \epsilon$

Result: String Accepted

Task 3:

Design a **Predictive Parser program** for the grammar:

$S \rightarrow aSb \mid \epsilon$

Test with strings:

ab

aabb

aaabbb

Explain parsing process step-by-step.

Solution:

FIRST and FOLLOW Sets:

FIRST(S) = {a, ε}

- $S \rightarrow aSb$ gives 'a'
- $S \rightarrow \epsilon$ gives ϵ

FOLLOW(S) = {b, \$}

- From $S \rightarrow aSb$, 'b' follows S
- \$ is end marker

Code:

```
#include <iostream>
#include <string>
using namespace std;
string input;
int index = 0;
void error() {
    cout << "\n String Rejected!" << endl;
    exit(0);
}
void match(char expected) {
    if (input[index] == expected) {
        index++;
        cout << "✓ Matched: " << expected << endl;
    } else {
        error();
    }
}
void S() {
    cout << "→ S called" << endl;

    if (input[index] == 'a') {
        // Apply production  $S \rightarrow a S b$ 
        cout << " Applying  $S \rightarrow a S b$ " << endl;
        match('a'); // Match 'a'
        S();        // Parse inner S
        match('b'); // Match 'b'
    }
    // else if (input[index] == 'b' || input[index] == '$') {
    else {
        // Apply production  $S \rightarrow \epsilon$ 
        cout << " Applying  $S \rightarrow \epsilon$ " << endl;
        // Do nothing (epsilon production)
    }
}
void printParseTree(int depth, string rule) {
    for(int i = 0; i < depth; i++) cout << " ";
    cout << rule << endl;
}
int main() {
    cout << "PREDICTIVE PARSER FOR GRAMMAR:  $S \rightarrow a S b \mid \epsilon$ " << endl;
    cout << "\nEnter string to parse (only a and b): ";
    cin >> input;
    input += "$"; // Add end marker
    cout << "\n--- Parsing Process ---\n" << endl;
    S(); // Start parsing
    if (input[index] == '$') {
        cout << "\n String Accepted!" << endl;
    } else {
        cout << "\n String Rejected!" << endl;
    }
}
```

```
}  
return 0;  
}
```

Output:

Testing ab:

```
PREDICTIVE PARSER FOR GRAMMAR:  $S \rightarrow a S b \mid \epsilon$   
  
Enter string to parse (only a and b): ab  
  
--- Parsing Process ---  
  
→ S called  
  Applying  $S \rightarrow a S b$   
✓ Matched: a  
→ S called  
  Applying  $S \rightarrow \epsilon$   
✓ Matched: b  
  
String Accepted!
```

Testing aabb:

```
PREDICTIVE PARSER FOR GRAMMAR:  $S \rightarrow a S b \mid \epsilon$   
  
Enter string to parse (only a and b): aabb  
  
--- Parsing Process ---  
  
→ S called  
  Applying  $S \rightarrow a S b$   
✓ Matched: a  
→ S called  
  Applying  $S \rightarrow a S b$   
✓ Matched: a  
→ S called  
  Applying  $S \rightarrow \epsilon$   
✓ Matched: b  
✓ Matched: b  
  
String Accepted!
```

Testing aaabbb:

```
PREDICTIVE PARSER FOR GRAMMAR:  $S \rightarrow a S b \mid \epsilon$   
  
Enter string to parse (only a and b): aaabbb  
  
--- Parsing Process ---  
  
→ S called  
  Applying  $S \rightarrow a S b$   
✓ Matched: a  
→ S called  
  Applying  $S \rightarrow a S b$   
✓ Matched: a  
→ S called  
  Applying  $S \rightarrow a S b$   
✓ Matched: a  
→ S called  
  Applying  $S \rightarrow \epsilon$   
✓ Matched: b  
✓ Matched: b  
✓ Matched: b  
  
String Accepted!
```

Rejected String:

```
PREDICTIVE PARSER FOR GRAMMAR:  $S \rightarrow a S b \mid \epsilon$ 

Enter string to parse (only a and b): aaabb

--- Parsing Process ---

→ S called
  Applying  $S \rightarrow a S b$ 
✓ Matched: a
→ S called
  Applying  $S \rightarrow a S b$ 
✓ Matched: a
→ S called
  Applying  $S \rightarrow a S b$ 
✓ Matched: a
→ S called
  Applying  $S \rightarrow \epsilon$ 
✓ Matched: b
✓ Matched: b

String Rejected!
```

Parsing Table:

Non-Terminal	a	b	\$
S	$S \rightarrow aSb$	$S \rightarrow \epsilon$	$S \rightarrow \epsilon$

Parsing Process Explanation:

For input "aabb":

1. **Start:** S is at top
2. **Step 1:** Look at 'a' → Apply $S \rightarrow aSb$
3. **Step 2:** Match 'a', now parse inner S with remaining "abb"
4. **Step 3:** Look at 'a' → Apply $S \rightarrow aSb$ again
5. **Step 4:** Match 'a', now parse inner S with "bb"
6. **Step 5:** Look at 'b' → Apply $S \rightarrow \epsilon$ (base case)
7. **Step 6:** Match 'b' from first production
8. **Step 7:** Match 'b' from second production
9. **Step 8:** All input consumed → Accepted

How the Parser Works:

1. **Predictive Parsing Table Lookup:**
 - At each step, look at current non-terminal and next input symbol
 - Use parsing table to determine which production to apply
2. **Production Rules:**
 - If next symbol is 'a': Use $S \rightarrow aSb$
 - If next symbol is 'b' or '\$': Use $S \rightarrow \epsilon$
3. **Stack Behaviour:**
 - The parser implicitly uses recursion (call stack)
 - Each call to S() handles one level of the parse tree
4. **Termination:**
 - Base case when no more 'a's to match ($S \rightarrow \epsilon$)
 - Then matches corresponding 'b's from each level